

CS6230 Project Report

Name: Kocherla Venkata Veerendra Babu Roll no.: EE25M102

Summary of RTL design using Bluespec SystemVerilog

Design of IEEE-754 single-precision floating point adder:

A struct is used to define the various fields of a 32-bit floating-point representation: the most significant bit (MSB) for the sign, the next 8 bits for the exponent, and the remaining 23 bits for the mantissa. Determining whether a number is zero is done by checking if both its mantissa and exponent are zero. Addition with zero is handled explicitly by returning the non-zero operand. If both operands are zero, the result is zero.

If both numbers are non-zero, the exponent difference is calculated and stored in `exp_diff`. The variables `mant_x` and `mant_y` are used to store the mantissas. An explicit 1 is appended to the MSB of both mantissas, and eight 0 bits are appended to the LSB to accommodate dropped bits, which are useful for even rounding. The mantissa corresponding to the smaller exponent is then shifted by the exponent difference. If the signs of both numbers are the same, their mantissas are added; otherwise, their difference is computed, and the sign of the result is assigned accordingly.

For rounding, I used the Round to Nearest, Ties to Even method. The fractional part of the mantissa was truncated to the first 23 bits following the MSB 01 (an additional 1'b0 was added at the MSB to handle any carry). The next bit is designated as the guard bit, followed by the round bit, and the XOR of all remaining bits in the LSB forms the sticky bit.

Rounding is performed if `(guard bit == 1) AND (round bit == 1 || sticky == 1) OR (lsb of extracted fraction == 1)`. During rounding, 32'b1 is added to the fraction, and the result is normalized accordingly. After normalization, the fraction is truncated to the required 23 bits (LSB). Finally, the sign, exponent (updated throughout the computation), and the truncated fraction are concatenated to form the final 32-bit floating-point representation of the sum of the given two 32-bit floating point numbers.

Design of IEEE-754 single-precision floating point Comparator:

NaN cases are handled by returning the other number as the output. If both operands are NaN, the result is a qNaN. For negative numbers, all bits are inverted to reverse their magnitude order, ensuring that more negative numbers appear "smaller" in unsigned comparisons. For positive numbers, the MSB is flipped. After these transformations, the numbers can be compared as normal unsigned values to yield the correct comparison result.

Design of Viterbi Decoder module:

All computations are performed within the **Viterbi decoder** module. The module takes two inputs, adds them, compares the result with the previously stored maximum value, and updates the stored value if the new result is greater. Simultaneously, the corresponding optimum state is updated. The module outputs the stored probability and the corresponding optimum state whenever the output method is called with the added emission matrix state.

The same mechanism is used during the **traceback** process. In this case, one input is set to zero and the other to the state probability. The **put** method is called repeatedly so that the module computes the maximum of all state probabilities. Whenever invoked, it outputs the maximum probability and the corresponding state.

- **Typedef Summary**

ViterbiState: FSM states controlling pipeline flow through addition, max comparison, and final probability computation phases.

- **Interface Summary**

lfc_viterbi: Methods to input transition/previous values, trigger final addition with emission probability, retrieve optimal state and final probability

- **Rule Summary**

rl_start_add: Initiates adder1 with transition and previous Viterbi values when both inputs ready; transitions to WaitAdd1

rl_get_add_result: Retrieves addition result, stores in add1_result, starts max comparison with current optprob; transitions to WaitMax

rl_get_max_result: Retrieves max result, updates optimal state if new maximum found, increments iteration counter; returns to Idle

rl_start_final_add: Initiates adder2 with optimal probability and emission value when start_final_add called; transitions to WaitAdd2

rl_get_final_result: Retrieves final addition result, stores temporarily; transitions to FinalAddReady for output retrieval

- **Method Summary**

put_a: Stores transition probability in val1 when idle and not already set

put_v: Stores previous Viterbi value in val2 when idle and not already set

get_optstate: Returns optimal state index when idle or final result ready

start_final_add: Triggers final probability computation by storing emission value and setting flag when idle

get_optprob: ActionValue method returning final probability, resets state machine and optprob to initial values for next iteration.

Design of Testbench module:

The testbench provides the correct values of the state matrix and the previous state probabilities on each clock cycle. The Viterbi module then performs addition, comparison, and storage of the correct probability along with its corresponding state. Whenever an emission matrix value is provided and the output method is invoked, the module returns the stored probability and corresponding state.

After each input sequence is processed, the previous memory is stored and reassigned. When the sequence terminates with 0xFFFFFFFF, the module initiates the traceback process. The traceback states are stored in registers and subsequently printed. Upon completion of all operations, all registers are cleared and reset.

- **Rule Summary**

rl_tick: Increments clock and prints cycle count every cycle

rl_open: Opens output file "Output_gen.dat" at startup

rl_loadnm: Loads N (states) and M (observations) from memory

rl_updots: Fetches next observation from input_mem and increments sequence pointer

rl_go: Processes current observation; calls Viterbi decoder with transition probabilities and previous Viterbi values

rl_wait_result: Single-cycle wait after Viterbi computation request

rl_start_final_add: Initiates final addition with emission probability (B matrix value)

rl_wait_final_add: Single-cycle wait for final addition

rl_capture: Stores optimal state and Viterbi probability; advances to next state or resets for next observation

rl_reset: Copies current Viterbi values to previous values array for next timestep

rl_recur: Finds maximum Viterbi value across all states for path probability calculation

rl_wait_recur: Single-cycle wait after recurrence computation

rl_start_path_add: Initiates path probability addition with zero

rl_wait_path_add: Single-cycle wait for path addition

rl_getpath: Retrieves final optimal state and path probability

rl_traceback: Reconstructs optimal state sequence by following backpointers in working_mem

rl_write: Writes traced states and path probability to output file

rl_clear: Resets all working memory arrays and state variables for next sequence

- **Typedef Summary**

State: FSM states controlling testbench flow through loading, Viterbi computation, traceback, and output phases.

All computations are made in Viterbi module and all correct indexing happens in Testbench module.

Summary of Optimization:

Initially, I implemented the find_leading_one function using a for loop with a static termination condition in version1. This function (with the for-loop) synthesizes to a priority encoder - a long chain of 33 cascaded multiplexers checking from bit 32 down to bit 0 sequentially. The critical path goes through all 33 levels in the worst case. Later I changed to the case statement that tells the synthesizer that conditions are mutually exclusive (only one bit can be the leading one), allowing it to use a parallel decoder structure rather than a sequential priority chain. This reduces logic depth significantly.

Initially, I performed 32-bit floating-point addition, maximum value computation, and 32-bit comparison for state updates within a single cycle (rule). This approach created a long chain of combinational logic blocks, leading to increased critical path delay. To improve performance, in version2, I separated these operations into different cycles(rules), ensuring that only one operation is executed per clock cycle. In version 3, Introduced FIFOs to manage handshaking efficiently and to ensure smooth data flow between stages.

The design was synthesized using Synopsys Design Compiler with the SAED32nm Educational Design Kit (EDK) standard cell library. Synthesis was performed using Low Voltage Threshold (LVT) cells under worst-case timing conditions: (slow-slow corner, 0.75V, 25°C).

Target library: saed32lvt_ss0p75v25c.db

PPA metrics obtained for different versions are:

Version	Clock period	Area	Power
Version 1	6.9ns	14535 μm^2	356.419 μW
Version 2	3.7ns	21867 μm^2	767.906 μW
Version 3	3.7ns	11654 μm^2	579.686 μW

Summary of Verification:

- ***viterbi.py***

Overview

A Python-based functional verification model was developed to validate the hardware implementation against a golden reference model. The verification environment processes identical test vectors and compares outputs bit-accurately with the Bluespec SystemVerilog synthesized design.

Implementation Architecture

The verification script implements the complete Viterbi algorithm using NumPy for efficient matrix operations. Key components include:

Input Processing:

- Reads design parameters (N states, M observations) and matrices from hexadecimal data files
- Converts IEEE 754 single-precision floating-point values from hexadecimal representation to Python float32 format using vectorized operations
- Parses observation sequences terminated by sentinel values (0xFFFFFFFF)

Viterbi Algorithm Core:

Algorithm performs forward pass computation:

1. Initialize path metrics using initial probabilities and first observation
2. For each time step t:
 - Compute branch metrics: $\text{prev_vt} + \text{transition_prob} + \text{emission_prob}$
 - Store traceback pointers (argmax of branch metrics)
 - Update path metrics (max of branch metrics)
3. Identify optimal final state (argmax of final path metrics)

Traceback and Output Generation:

- Backward traversal through stored traceback pointers recovers optimal state sequence
- Converts state indices to 1-indexed hexadecimal format matching hardware output convention
- Outputs optimal probability in IEEE 754 hex format for bit-accurate comparison

It writes result into a file and prints the address of the saved output file.

- ***compare.py***

Compares two files provided to it and prints those lines where they are different and outputs total number of mismatched lines.

- ***create_testcase.py***

Creates random testcases. Provide the required values. For any parameter, pressing Enter without input triggers automatic random value selection within valid ranges. Make sure that the values you choose will not result in any 'dat' file exceeding 1024 lines (constraint provided in the problem statement). Also note that maximum memory size allowed is 1000 (constraint provided in the problem statement), so number of states * length of input sequence cannot be more than 1000.

Each generated test case is automatically validated against both constraints before file generation. Invalid parameter combinations trigger re-selection to ensure compliance with hardware limitations.

The generator produces four files compatible with the hardware testbench:

- N.dat: State and observation counts (N, M)
- A.dat: Transition probability matrix in IEEE 754 hex format
- B.dat: Emission probability matrix in IEEE 754 hex format
- input.dat: Observation sequences with 'FFFFFFFF' delimiters

It also prints the folder path in which the custom-test-case is saved.